
Discovering Cooperative Pipelines: Autoresearch for Sequential Social Dilemmas

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study *two-level autoresearch*: an outer-loop AI agent that autonomously re-
2 designs the inner-loop pipeline of an LLM policy-synthesis system for multi-agent
3 Sequential Social Dilemmas (SSDs). A *researcher agent* $\mathcal{R}$ (Claude Opus 4.6,
4 run as a coding agent) reads the inner-loop source code, edits system prompts,
5 feedback functions, helper libraries, and iteration logic, runs evaluations, and de-
6 cides what to keep, following the *autoresearch* paradigm. Across 12 autonomous
7 runs spanning two games (Cleanup and Gathering), two policy-synthesizer LLMs
8 (Claude Sonnet 4.6, Gemini 3.1 Pro), and two welfare objectives (utilitarian ef-
9 ficiency and Rawlsian maximin), the researcher reliably exceeds hand-designed
10 baselines, sharply tightens run-to-run variance, and substantially outperforms
11 prompt-only optimization (GEPA), which cannot overcome the stochasticity of
12 LLM code generation. The discovered pipelines are genuinely a function of the
13 welfare objective: under maximin the researcher independently rediscovers *duty*
14 *rotation*—never found under efficiency optimization—supporting an *information-*
15 *design* interpretation in which the researcher chooses what to expose to the bound-
16 edly rational synthesizer. The Cleanup–Gathering contrast further reveals that
17 *game structure* dictates whether fairness needs explicit optimization: in *provi-*
18 *sion* dilemmas it requires designed mechanisms, while in *restraint* dilemmas
19 it emerges as a byproduct of coordination. Code at [https://github.com/](https://github.com/anon590/llm-policies-social-dilemmas)
20 [anon590/llm-policies-social-dilemmas](https://github.com/anon590/llm-policies-social-dilemmas).

21 1 Introduction

22 Sequential Social Dilemmas (SSDs) [1] are the multi-agent analogue of the prisoner’s dilemma
23 in temporally rich Markov games: individually rational play leads to collectively suboptimal out-
24 comes through pollution, over-harvesting, or open conflict. Standard multi-agent reinforcement
25 learning (MARL) struggles in this regime due to credit assignment, non-stationarity, and large
26 joint action spaces [2]. A complementary approach, recently introduced by Gallego [3], replaces
27 parameter-space optimization with *algorithm-space synthesis*: a frozen LLM writes a Python policy
28 function, evaluates it in self-play, and iteratively refines it from performance feedback. A single
29 generation step can produce coordination logic—territory partitioning, role assignment, conditional
30 cooperation—that would take millions of RL episodes to discover.

31 This shifts where the design problem lives, rather than removing it. The inner-loop pipeline that
32 drives the synthesizer has many free parameters: which system prompt, which feedback variables,
33 which helper functions, how many refinement steps. Each materially affects the resulting policies,
34 and prior work tuned them by hand. A natural question follows: *can an AI agent design the pipeline?*

35 We answer affirmatively with a two-level autoresearch framework. An *outer-loop researcher agent*
36 (Claude Opus 4.6, run as a coding agent via the Claude Code CLI) edits the source files of an *inner-*

37 *loop policy synthesizer* (Gemini 3.1 Pro or Claude Sonnet 4.6), runs evaluations on held-out seeds,
 38 and keeps modifications that improve a fixed welfare objective Φ . The outer agent operates on an
 39 ordinary git repository—reading code, writing diffs, running shell commands—without task-specific
 40 scaffolding, mirroring the autoresearch paradigm of Karpathy [4] for single-GPU LLM pretraining.
 41 Although the inner-loop SSDs are gridworld benchmarks rather than physical systems, the outer-
 42 loop discovery process itself runs under conditions a deployed discovery agent faces: noisy multi-
 43 seed evaluations, stochastic code generation, an LLM-evaluation budget that bounds how often Φ
 44 can be queried, and a heterogeneous Python repository the agent must navigate end-to-end on its
 45 own.

46 Our contributions are:

- 47 • A general two-level framework that delegates the design of an LLM synthesis pipeline to a coding
 48 agent operating on a real software repository (Section 3).
- 49 • The first instantiation of the autoresearch paradigm in a multi-agent decision-making domain, with
 50 12 autonomous runs across two SSDs (Cleanup and Gathering), two policy LLMs, and two welfare
 51 objectives (utilitarian efficiency U and Rawlsian maximin $\min_i R_i$). The researcher reliably ex-
 52 ceeds hand-designed baselines and dominates prompt-only optimization (GEPA) [5] (Section 4).
- 53 • A mechanism-design interpretation supported by the qualitatively different pipelines the agent
 54 produces under different welfare objectives—including the autonomous discovery of *time-based*
 55 *duty rotation* as a fairness mechanism, never found under efficiency optimization.

56 2 Background

57 We build on the iterative LLM policy synthesis framework of Gallego [3], which serves as the
 58 (frozen) inner loop of our two-level system. This section recalls the SSD formalism, the social
 59 outcome metrics, and the base synthesis loop.

60 2.1 Sequential Social Dilemmas

61 A Sequential Social Dilemma is a partially observable Markov game $\mathcal{G} =$
 62 $\langle N, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$ with N agents, state space $\mathcal{S}$ (the gridworld configuration),
 63 per-agent action spaces $\mathcal{A}_i$, transition function T , reward functions R_i , and episode horizon H [1].
 64 Beyond the dilemma’s matrix-game structure, SSDs add temporal richness: agents must learn *when*
 65 and *where* to cooperate, not just *whether* to.

66 We study two canonical SSDs that capture complementary dilemma types.

67 **Cleanup** [6] is a *public goods provision* game ($N=10$). The map has two regions: a river that
 68 accumulates waste, and an orchard where apples grow. Apples regrow only when river pollution is
 69 below threshold. Each agent can fire a cleaning beam (cost -1) that removes waste, collect apples
 70 ($+1$ each), or fire a tagging beam (cost -1 , inflicting -50 on the target and removing it for 25 steps).
 71 The dilemma: cleaning is costly but its benefits are public, so purely self-interested agents free-ride.

72 **Gathering** [1, 7] is a *common pool resource* game ($N=4$). Agents collect shared apples on a fixed
 73 respawn timer and may fire tagging beams to temporarily remove rivals. The dilemma: agents can
 74 coexist and share resources, or aggress to monopolize them; aggression wastes time and reduces
 75 total welfare.

76 Both games use 8–9 discrete actions (movement, rotation, beam, stand, optionally clean) and
 77 episodes of $H=1000$ steps. The two dilemmas differ in cost structure—asymmetric provision (clean-
 78 ers pay, all benefit) vs. symmetric restraint (every agent faces the same temptation)—a distinction
 79 that drives our key findings.

80 **Social metrics.** Following Perolat et al. [7], let $R_i = \sum_{t=0}^{H-1} r_i^t$ denote agent i 's episode return.
 81 We evaluate four social outcomes:

$$U = \frac{1}{H} \sum_{i=1}^N R_i \quad (\text{efficiency}) \quad (1)$$

$$E = 1 - \frac{\sum_{i,j} |R_i - R_j|}{2N \sum_i R_i} \quad (\text{equality}) \quad (2)$$

$$S = \frac{1}{N} \sum_{i=1}^N \bar{t}_i \quad (\text{sustainability}) \quad (3)$$

$$P = \frac{1}{H} \sum_{t=0}^{H-1} |\{i : \text{active}_i^t\}| \quad (\text{peace}) \quad (4)$$

82 where $\bar{t}_i$ is the mean timestep at which agent i collects positive reward (higher $\Rightarrow$ resources preserved
 83 later) and active_i^t indicates that agent i is not tagged out at step t . We additionally consider the
 84 *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which optimizes the worst-off agent's return and
 85 serves as the second objective in our experiments.

86 2.2 Iterative LLM Policy Synthesis

87 Let Π denote the space of *programmable policies*: deterministic functions $\pi : \mathcal{S} \times [N] \rightarrow \mathcal{A}$ ex-
 88 pressed as executable Python code. Each policy has access to the full environment state and a
 89 library of helpers (BFS pathfinding, beam targeting, coordinate transforms). This state access is a
 90 deliberate design choice: programmatic policies operate in algorithm space rather than the reactive
 91 observation-to-action space of neural policies, which lets a single LLM generation step encode rich
 92 coordination logic.

93 A frozen LLM $\mathcal{M}$ acts as the *policy synthesizer*. Given a system prompt p describing the environ-
 94 ment API and a feedback prompt q_k , it produces a new policy

$$\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k)),$$

95 where π_k is the previous policy (its source code) and $\mathcal{F}_k$ is the evaluation feedback. All N agents ex-
 96 ecute the same policy in homogeneous self-play, and evaluation over a set of random seeds S yields
 97 the mean per-agent return $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$. Each generated
 98 policy passes an AST-based safety check (blocking eval, file I/O, network access) followed by a
 99 short smoke test; failures trigger regeneration (up to R attempts) with the error message appended
 100 to the prompt.

101 **Feedback.** The hand-designed feedback used by Gallego [3] as our starting point packages the
 102 previous policy's source code together with all available evaluation signals:

$$\mathcal{F}_k = (\text{code}(\pi_k), \bar{r}_k, \mathbf{m}_k, \mathbf{d}), \quad (5)$$

103 where $\mathbf{d}$ contains natural-language definitions of each social metric. Exposing the full social-metrics
 104 vector (rather than the scalar reward alone) disambiguates distinct failure modes that share the same
 105 average return—e.g., under- vs. over-cleaning in Cleanup—and consistently improves cooperative
 106 outcomes. On Cleanup, this baseline feedback reaches $U=1.93/2.70$ (mean/max) with Gemini and
 107 $U=0.86/1.56$ with Sonnet across 4 runs each. The choice of feedback content is a single design
 108 decision within a much larger pipeline configuration space; our two-level framework (Section 3)
 109 opens the full space to automated search.

110 3 Two-Level Framework

111 We introduce a two-level system where a *researcher agent* $\mathcal{R}$ autonomously discovers configurations
 112 that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy
 113 synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates them,
 114 and iterates can serve as the inner loop. The key insight is that the entire inner-loop codebase—not
 115 just the feedback level—is a designable artifact that an AI agent can search over. Figure 1 illustrates
 116 the architecture.

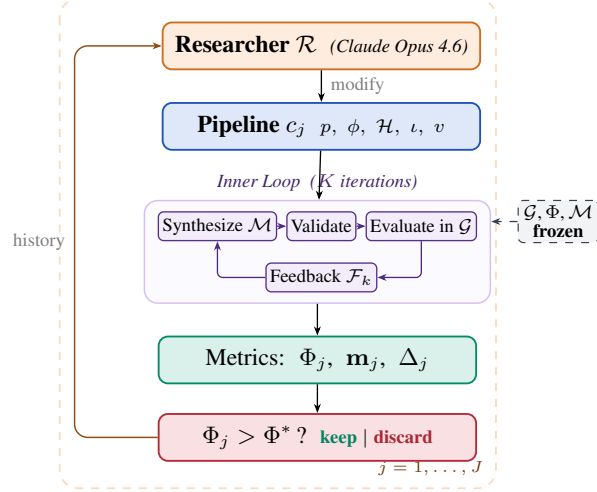


Figure 1: Two-level automated research framework (Algorithm 1). *Outer loop*: the researcher $\mathcal{R}$ modifies the pipeline configuration c_j , runs the inner loop, and keeps or discards based on Φ . *Inner loop*: the policy LLM $\mathcal{M}$ iteratively synthesizes, validates, and evaluates policies in the frozen environment $\mathcal{G}$.

Table 1: Configuration components modifiable by the researcher. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

	Scope	Examples
p	System prompt	Hints, worked examples
ϕ	Feedback fn	Metric subset, framing, derived metrics, trends
$\mathcal{H}$	Helper library	Pathfinding, zones
ι	Iteration logic	K , eval seeds, best-of- n
v	Validation	Safety checks, smoke tests

117 3.1 Configuration Space

118 Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-
119 loop setup:

$$c = (p, \phi, \mathcal{H}, \iota, v) \quad (6)$$

120 where p is the system prompt, ϕ is the feedback construction function (which content to include, how
121 to frame it, whether to inject adaptive hints), $\mathcal{H}$ is the helper function library, ι specifies the iteration
122 logic (number of iterations K , sampling strategy, temperature schedule), and v is the validation
123 pipeline. Table 1 provides concrete examples.

124 The hand-designed feedback of [3] corresponds to a single fixed instantiation of ϕ . Our framework
125 opens the full configuration space to automated search.

126 3.2 Inner Loop (Policy Synthesis)

127 Given a configuration c , the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c) \quad (7)$$

128 Each iteration k proceeds in four stages, following [3]:

- 129 1. **Synthesize.** The policy LLM $\mathcal{M}$ receives the system prompt p , the previous policy’s source code
130 π_{k-1} , and feedback $\mathcal{F}_{k-1}$ constructed by ϕ . It generates a new Python policy function π_k that
131 has access to full environment state and the helper library $\mathcal{H}$.
- 132 2. **Validate.** The generated code undergoes AST-based safety checks (blocking dangerous oper-
133 ations such as file I/O and network access) followed by a short smoke test. Failures trigger
134 re-generation (up to R retries), with the error message appended to the prompt.

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt for researcher $p_{\mathcal{R}}$, initial configuration c_0 , outer iterations J , ground-truth objective Φ , held-out seeds $S_{\text{held-out}}$

Ensure: Best configuration c^* , best policy π^*

```
1:  $\pi_0^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_0)$ 
2:  $\Phi_0 \leftarrow \text{EVAL}(\pi_0^*, \mathcal{G}, S_{\text{held-out}})$ 
3:  $history \leftarrow \{(c_0, \Phi_0, \mathbf{m}_0, \emptyset)\}$ 
4: for  $j = 1, \dots, J$  do
5:    $c_j \leftarrow \mathcal{R}(p_{\mathcal{R}}, \text{CODE}(c_{j-1}), history)$ 
6:   if  $\neg \text{VALIDATECONFIG}(c_j)$  then retry  $\leq R$  times
7:    $\pi_j^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_j)$ 
8:    $\Phi_j \leftarrow \text{EVAL}(\pi_j^*, \mathcal{G}, S_{\text{held-out}})$ 
9:    $\Delta_j \leftarrow \text{DIFF}(c_{j-1}, c_j)$  // code diff
10:   $history \leftarrow history \cup \{(c_j, \Phi_j, \mathbf{m}_j, \Delta_j)\}$ 
11: end for
12:  $j^* \leftarrow \arg \max_j \Phi_j$ 
13: return  $c_{j^*}, \pi_{j^*}^*$ 
```

- 135 3. **Evaluate.** All N agents execute the same policy π_k in self-play over $|S|$ random seeds. The eval-
136 uation yields the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$.
137 4. **Feedback.** The feedback function ϕ constructs the prompt for the next iteration from
138 $(\pi_k, \bar{r}_k, \mathbf{m}_k)$, packaging the previous policy’s source code together with the scalar reward, the
139 social metrics vector, their natural-language definitions, and any adaptive diagnostics that ϕ in-
140 jects.

141 The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \text{EVAL}(\pi_c^*, \mathcal{G}, S_{\text{held-out}}) \quad (8)$$

142 where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \quad (\text{utilitarian efficiency}) \quad (9)$$

$$\Phi_{\min} = \min_i R_i \quad (\text{Rawlsian maximin}) \quad (10)$$

143 3.3 Outer Loop (Automated Research)

144 The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch
145 paradigm [4], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes,
146 observing outcomes, and refining. The procedure is formalized in Algorithm 1.

147 At each outer iteration j , the researcher $\mathcal{R}$ receives:

- 148 • The **full source code** of the current configuration c_{j-1} (prompts, feedback construction, helpers,
149 iteration logic).
- 150 • The **experiment history**: for each prior iteration, the code diff Δ_i , ground-truth score Φ_i , and
151 social metrics vector $\mathbf{m}_i$.
- 152 • The **environment source code** (read-only), enabling the researcher to reason about game mechan-
153 ics.

154 The researcher proposes a new configuration c_j by generating code modifications. Concretely, $\mathcal{R}$
155 is a coding agent (Claude Code CLI) that operates on a real software repository: it reads and edits
156 Python source files, runs shell commands, inspects evaluation outputs, and commits changes to
157 a dedicated git branch—the same workflow a human researcher would follow. This mirrors the
158 autoresearch setup, where an AI coding agent modifies `train.py` and observes validation loss;
159 here, the researcher modifies the inner-loop codebase and observes social welfare.

160 3.4 Connection to Automated Mechanism Design

161 The two-level structure admits a mechanism design interpretation. The researcher $\mathcal{R}$ acts as a *mech-*
162 *anism designer*: it controls the information structure (what metrics to reveal, how to frame them),

the action space (what helper functions are available), and the incentive structure (how feedback is presented) under which the policy synthesizer $\mathcal{M}$ operates. The synthesizer acts as the *agent* within the designed mechanism.

This connects to the automated mechanism design literature [8], where a principal designs rules to induce desired behavior from self-interested agents. In our setting:

- The **principal** is the researcher $\mathcal{R}$, optimizing the ground-truth objective Φ .
- The **agent** is the synthesizer $\mathcal{M}$, optimizing per-agent reward as instructed.
- The **mechanism** is the configuration c : prompts, feedback, helpers, iteration logic.
- The **outcome** is the social welfare of the resulting multi-agent policy π_c^* .

A key distinction from classical mechanism design: $\mathcal{M}$ is not *strategically* self-interested—it follows instructions—but it has *bounded rationality* in the sense that its ability to synthesize effective policies depends on the information and tools provided. The researcher’s task is thus closer to *information design* [9]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments (Section 4) show that the researcher designs qualitatively different information structures depending on the welfare objective Φ , supporting this interpretation empirically.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase: $\mathcal{R}$ edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates—without human intervention.

Design. For Cleanup: 2 policy LLMs $\times$ 2 objectives $\times$ 2 replications = 8 runs. For Gathering: 2 policy LLMs $\times$ 1 objective $\times$ 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game’s symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [3]. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework’s behavior when the policy synthesizer has substantially lower capability (Appendix C).

Baselines. The hand-designed feedback configuration from [3] serves as the initial pipeline c_0 for all runs. On Cleanup ($N=10$), this baseline achieves $U=1.93/2.70$ (mean/max) with Gemini and $U=0.86/1.56$ with Sonnet across 4 runs each. We additionally compare against GEPA [5], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt p , whereas our researcher modifies the full pipeline $(p, \phi, \mathcal{H}, \iota, v)$. We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method (each GEPA step costs ~ 3 policy evaluations, comparable to our inner loop’s $K+1=4$).

Inner loop parameters. $K=3$ synthesis iterations (or $K=2$ if the researcher elects), evaluated over $|S|=5$ –12 random seeds (researcher-configurable). The researcher ran $J=3$ –17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 2 presents the main results, comparing autoresearch against the hand-designed baseline of [3] and GEPA prompt optimization [5].

Finding 1: The researcher reliably improves over hand-designed baselines and outperforms prompt-only optimization. All 12 runs show substantial improvement regardless of starting point (Figure 2). In Cleanup, the researcher exceeds the unmodified pipeline baseline by 66% with Gemini ($U: 1.93 \rightarrow 3.20$) and 263% with Sonnet ($U: 0.86 \rightarrow 3.12$). Strikingly, the Sonnet improvement

Table 2: Results. Mean / max across 2 runs per condition (4 for Cleanup baselines). Baseline: unmodified pipeline from [3]. GEPA: automated prompt optimization [5] with matched compute budget.

Policy LLM	Target	U	E	$\min_i R_i$
<i>Cleanup (N=10) — Baseline</i>				
Gemini 3.1 Pro	—	1.93/2.70	0.17/0.62	−159/−84
Sonnet 4.6	—	0.86/1.56	−0.02/0.25	−151/−59
<i>Cleanup (N=10) — GEPA [5]</i>				
Gemini 3.1 Pro	Φ_U	1.34/1.37	−0.10/−0.05	−149/−126
	$\Phi_{\min}$	1.76/2.76	0.90/0.96	143/245
Sonnet 4.6	Φ_U	1.04/1.20	−0.59/−0.21	−164/−126
	$\Phi_{\min}$	−0.63/1.60	0.77/1.00	−147/6
<i>Cleanup (N=10) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	3.20 /3.25	0.55/0.61	−211/−182
	$\Phi_{\min}$	3.16/3.19	0.98 /0.98	290 /296
Sonnet 4.6	Φ_U	3.12 /3.14	0.66/0.70	−196/−146
	$\Phi_{\min}$	2.57/2.93	0.91 /0.97	179 /200
<i>Gathering (N=4) — Baseline</i>				
Gemini 3.1 Pro	—	2.04/2.42	0.90/0.98	412/571
Sonnet 4.6	—	0.03/0.03	0.54/0.54	0/0
<i>Gathering (N=4) — GEPA [5]</i>				
Gemini 3.1 Pro	Φ_U	2.08/2.35	0.94/0.96	436/518
Sonnet 4.6	Φ_U	1.20/1.23	0.63/0.71	86/123
<i>Gathering (N=4) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	2.49/2.51	0.98/0.98	582/582
Sonnet 4.6	Φ_U	2.52/2.52	0.96/0.98	576/597

210 nearly closes the gap with Gemini ($U=3.12$ vs. 3.20), despite a $2\times$ baseline disadvantage (0.86 vs.
211 1.93). Final efficiency values cluster tightly around $U \approx 3.1$ –3.2 despite baselines varying from
212 0.29 to 2.70 across individual runs, suggesting the researcher reliably discovers the performance
213 ceiling of each policy LLM. Beyond improving the mean, the researcher yields remarkably consistent
214 results: under efficiency optimization, Gemini’s runs cluster within $U=3.20$ –3.25 (gap 0.05)
215 despite baselines spanning 1.15–2.70, and Sonnet’s within 3.12–3.14 (gap 0.02) from baselines
216 of 0.38–1.56. In Gathering, all four runs converge to $U=2.47$ –2.52 from baselines ranging 0.03–
217 2.42. The researcher discovers pipeline modifications—structured helpers (App. D.1), strategic hints
218 (App. D.2), tailored feedback (App. D.3)—that constrain the policy LLM toward consistently good
219 solutions.

220 Compared to GEPA (Table 2), autoresearch achieves $2.4\times$ higher efficiency under Φ_U in Cleanup
221 ($U=3.20$ vs. 1.34), $2\times$ higher maximin under $\Phi_{\min}$ (290 vs. 143), and 20% higher efficiency in Gather-
222 ing ($U=2.49$ vs. 2.08). Critically, GEPA shows far wider spread across runs on the harder fairness
223 objective ($\Phi_{\min}$ maximin in Cleanup: 245/41 vs. autoresearch’s 296/284). The gap widens further
224 with Sonnet: on Cleanup Φ_U , GEPA stalls at $U=1.04$ ($3\times$ below autoresearch’s 3.12), and on $\Phi_{\min}$
225 its two runs split between a modestly positive solution ($\min_i R_i=6$, $U=1.60$) and a pathological
226 “everyone cleans, nobody eats” regime (perfect equality $E=1.00$ but negative efficiency $U=−2.87$
227 and $\min_i R_i=−299$), while autoresearch with the same model reaches $\min_i R_i=179/200$ reliably.
228 This highlights the advantage of modifying the full pipeline—helpers, feedback, and iteration logic
229 provide structured scaffolding that reduces run-to-run variance, whereas prompt-only optimization
230 cannot address the underlying stochasticity of code generation.

231 **Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini).** Maximin-optimized Gem-
232 ini pipelines sacrifice only 1% efficiency (U : 3.16 vs. 3.20) while achieving near-perfect equal-
233 ity (E : 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (−99 to −84) to
234 $\min_i R_i = 290$. The researcher discovers *fair duty rotation* (Listing 6; primed by the prompt rewrite
235 of Listing 3)—time-based cycling using `agent_id` and `env._step_count`—simultaneously im-
236 proving worst-off welfare *and* collective output. Because cleaning is a public good, distributing the
237 cleaning cost fairly ensures enough cleaners to sustain apple production.

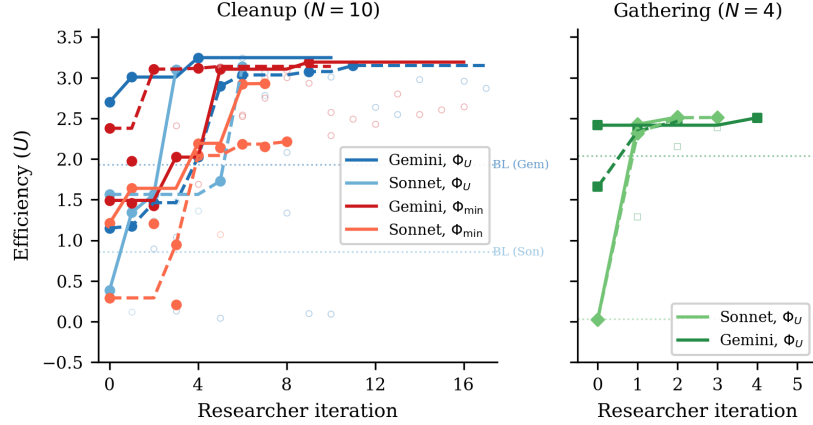


Figure 2: Efficiency (U) across researcher iterations for all 12 runs. *Left*: Cleanup ($N=10$) with 8 runs across 2 LLMs $\times$ 2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [3]. All runs converge to $U \approx 3.1$ – 3.2 despite diverse starting points. *Right*: Gathering ($N=4$) with 4 runs (2 per LLM). All converge to $U \approx 2.5$ within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin optimization. The gap reflects Sonnet’s harder time implementing complex coordination mechanisms (role rotation, zone assignment) from strategic hints alone.

Finding 3: Game structure determines whether fairness requires explicit optimization. In Cleanup, where cleaning costs are borne asymmetrically (cleaners pay -1 , free-riders collect apples), baseline equality ranges from $E=0.04$ to 0.62 , and maximin optimization is required to reach $E > 0.9$. In Gathering, where all agents face a symmetric landscape, efficiency optimization alone achieves $E > 0.94$ across all 4 runs—no separate maximin runs are needed.

This generalizes: in *provision* dilemmas with asymmetric costs, fairness requires designed mechanisms (role rotation, duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a free byproduct of efficient coordination. The researcher independently discovers this—it creates role differentiation pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

Finding 4: Convergent discovery of qualitatively different strategies per objective. Despite fully independent runs, the researcher converges on the same core strategies within each condition (Table 3, Appendix A). In Cleanup, waste-counting helpers and spatial zone partitioning appear across all 8 runs. The key qualitative difference is *role rotation*, appearing in 4/4 maximin runs (Listings 6, 7) but 0/4 efficiency runs (Listing 5). Under efficiency optimization, the researcher assigns static cleaning roles (some agents always clean), achieving high collective output at the cost of equality. Under maximin optimization, it discovers rotation as a fairness mechanism.

In Gathering, the researcher discovers BFS-Voronoi territory partitioning and respawn-timer awareness (Listings 2, 8), with no role differentiation—optimization is purely spatial (who collects which apples) and temporal (respawn-aware positioning).

Common failure modes. Several patterns led to discarded experiments across all runs: (1) *over-prescription*—adding too many strategic hints confuses the policy LLM, causing generation failures; (2) *iteration regression*— $K=4$ or $K=5$ inner iterations often cause catastrophic regression as the LLM “over-refines” a working policy; (3) *feedback overload*—verbose per-agent diagnostics cause over-correction rather than gradual improvement; (4) *variance exposure*—identical configurations can yield $U=3.2$ then $U=0.09$ on different seeds, requiring multi-seed evaluation for stable signals (per-run K , $|S|$ in Table 6).

5 Related Work

Sequential social dilemmas. SSDs were introduced by Leibo et al. [1] (Gathering) and extended to public-goods settings by Hughes et al. [6] (Cleanup); Perolat et al. [7] formalized the social outcome metrics (U, E, S, P) used here. Subsequent work studies inequity aversion, intrinsic motivation, and reputational mechanisms for promoting cooperation in MARL agents. We complement this line by automating the search for cooperative *programs* rather than evolving neural policies, and by treating the welfare objective Φ as a designable parameter that the outer agent optimizes for—obtaining qualitatively different cooperative behaviors (static role assignment vs. duty rotation) under different objectives without changing the environment.

LLMs for policy and program synthesis. FunSearch [10] evolves programs for combinatorial discovery; Eureka [11] synthesizes reward functions from environment source code; Voyager [12] and Code as Policies [13] generate executable skill code for single-agent embodied control; ReEvo [14] evolves heuristics with reflective feedback. These works target single-agent settings or non-strategic optimization. Gallego [3], on which our inner loop is based, extended LLM program synthesis to the multi-agent SSD setting, where one program must coordinate N self-play copies. Our contribution is one level above: rather than tuning the synthesizer for one task, we let an outer agent rewrite the synthesis pipeline itself.

LLM reflection and prompt optimization. Reflexion [15], Self-Refine [16], OPRO [17], and GEPA [5] demonstrate that structured verbal feedback loops improve LLM outputs; ERL [18] internalizes such reflection via self-distillation. These methods optimize the prompt or the model’s reasoning trajectory in isolation. Our framework instead modifies the entire surrounding pipeline—system prompt, feedback construction, helper library, iteration logic, validation—making prompt optimization (cf. GEPA) one component of a strictly larger search space. The empirical gap is large: Section 4 shows full-pipeline autoresearch achieving $3\times$ higher efficiency in Cleanup and 37% higher in Gathering than GEPA at matched compute, with run-to-run spread an order of magnitude tighter on the harder fairness objective.

Automated AI research. A small but growing body of work delegates the design of ML pipelines to LLM coding agents. Karpathy’s autoresearch [4] runs a coding agent that modifies `train.py` for nanoGPT pretraining and is rewarded for validation loss. Our system shares this autoresearch architecture—coding agent + frozen evaluation harness + diff-based history—but targets a different inner loop (multi-agent policy synthesis instead of single-model pretraining) and a different objective (multi-agent social welfare instead of validation bits-per-byte). To our knowledge this is the first instantiation of the autoresearch paradigm in a multi-agent decision-making domain.

Automated mechanism and information design. Classical automated mechanism design [8] computes optimal allocation rules for self-interested agents under explicit incentive constraints, while information design [9] studies how a principal commits to a signaling policy that shapes a receiver’s behavior. Our outer agent solves a related but distinct problem: $\mathcal{M}$ is not strategically deceptive (it follows instructions) but is *boundedly rational*, so the researcher must decide what information, helpers, and structure to expose so that $\mathcal{M}$ writes policies achieving the principal’s welfare goal. Section 4 shows that the pipelines $\mathcal{R}$ produces are genuinely a function of the welfare objective, supporting this information-design framing empirically.

6 Discussion and Conclusion

We have presented a two-level framework where an AI coding agent autonomously discovers pipeline configurations that improve the output of an inner-loop LLM system. Applied to multi-agent policy synthesis in social dilemmas, the researcher agent reliably exceeds hand-designed baselines across 12 independent runs, and converges on qualitatively similar strategies within each game-objective condition. The framework requires no domain-specific scaffolding: the agent operates on a standard software repository using the same tools (file editing, shell commands, git) available to a human researcher.

Mechanism design in action. Our results empirically support the mechanism design interpretation of Section 3.4. The researcher acts as an information designer: under Φ_U , it reveals efficiency-oriented information (waste counts, zone assignments) that guides $\mathcal{M}$ toward productive but unequal role allocation (Listing 5). Under $\Phi_{\min}$, it additionally reveals fairness-oriented structure—rotation schedules and per-agent equity feedback (Listings 3, 4)—guiding $\mathcal{M}$ toward egalitarian coordination. The striking absence of a fairness–efficiency tradeoff with Gemini suggests that in public goods provision, the mechanism design problem has a cooperative optimum where both objectives align—the researcher discovers this independently.

Two types of dilemma. The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent’s contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent—suggesting it is a robust structural feature of these environments.

Human oversight and audit. Our system is fully autonomous by design, but its architecture offers natural affordances for human-in-the-loop oversight. Because the researcher $\mathcal{R}$ operates on a standard git repository—committing each pipeline modification to a dedicated branch—a domain expert can audit the full history of code diffs Δ_j alongside their metric outcomes $(\Phi_j, \mathbf{m}_j)$, exactly as they would review a colleague’s pull requests. The keep/discard decision (Algorithm 1, line 11) is a lightweight intervention point: a human could override the accept threshold, veto modifications that introduce undesirable mechanisms (e.g., exploitative role assignments), or steer the researcher toward under-explored regions of the configuration space by editing the experiment history. More broadly, the separation between the researcher $\mathcal{R}$ and the frozen evaluation Φ creates a *delegation boundary*: the human defines *what* to optimize (the welfare objective), while the agent decides *how*. This mirrors the expert-in-the-loop design patterns identified in deployment-oriented work on AI agents for discovery, where trust calibration depends on the legibility of the agent’s decision trail rather than on real-time supervision.

Limitations. Our experiments use only 2 replications per condition, yielding high uncertainty on some estimates (e.g., Sonnet $\Phi_{\min}$: $U=2.57/2.93$). The two SSDs are small gridworld environments; scaling to larger, more complex games remains open. The researcher runs are not fully controlled—the number of outer iterations varies from 3 to 17 (mean 8.3)—making direct comparison across conditions approximate. The GEPA comparison uses a single run per condition; additional replications would strengthen confidence in the comparison.

Future work. Five directions emerge: (1) *new domains*—apply the two-level framework to other LLM-driven pipelines (code optimization, scientific experiment design, infrastructure tuning), where the same architecture applies with a different inner loop and objective Φ ; (2) *ablation*—freeze all configuration components except one to isolate the relative importance of prompt, feedback, helpers, and iteration logic; (3) *adversarial objectives*—set $\Phi = R_0$ (agent 0’s reward only) to test whether the researcher discovers exploitative pipeline configurations, connecting to the reward hacking analysis of [3]; (4) *cross-game transfer*—run the best Cleanup configuration on Gathering and vice versa, testing game-generalizability; (5) *heterogeneous policies*—extend to settings where agents run different code, enabling richer role specialization. Code is available at <https://github.com/vicgalle/llm-policies-social-dilemmas>.

References

- [1] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*, pages 464–473, 2017.
- [2] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [3] Víctor Gallego. Cooperation and exploitation in LLM policy synthesis for sequential social dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*, 2026.

- 369 [4] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training
370 automatically. <https://github.com/karpathy/autoresearch>, 2026.
- 371 [5] Lakshya A Agrawal et al. GEPA: Reflective prompt evolution can outperform reinforcement
372 learning. *arXiv preprint arXiv:2507.19457*, 2026.
- 373 [6] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Anto-
374 nio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and
375 Thore Graepel. Inequity aversion improves cooperation in intertemporal social dilemmas. In
376 *Neural Information Processing Systems*, 2018.
- 377 [7] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Grae-
378 pel. A multi-agent reinforcement learning model of common-pool resource appropriation. In
379 *Advances in Neural Information Processing Systems*, volume 30, 2017.
- 380 [8] Vincent Conitzer and Tuomas Sandholm. Complexity of mechanism design. In *Proc. 18th*
381 *Conference on Uncertainty in Artificial Intelligence*, pages 103–110, 2002.
- 382 [9] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*,
383 101(6):2590–2615, 2011.
- 384 [10] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog,
385 M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang,
386 Omar Fawzi, et al. Mathematical discoveries from program search with large language models.
387 *Nature*, 625(7995):468–475, 2024.
- 388 [11] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh
389 Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward
390 design via coding large language models. In *The Twelfth International Conference on Learning*
391 *Representations*, 2024.
- 392 [12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi
393 Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language
394 models. *Transactions on Machine Learning Research*, 2024.
- 395 [13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence,
396 and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023*
397 *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.
- 398 [14] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo
399 Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective
400 evolution. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*,
401 2024.
- 402 [15] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Re-
403 flexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on*
404 *Neural Information Processing Systems*, 2023.
- 405 [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe,
406 Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative re-
407 finement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing*
408 *Systems*, 2023.
- 409 [17] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun
410 Chen. Large language models as optimizers. In *The Twelfth International Conference on*
411 *Learning Representations*, 2024.
- 412 [18] Taiwei Shi, Sihao Chen, Bowen Jiang, Linxin Song, Longqi Yang, and Jieyu Zhao. Experiential
413 reinforcement learning. *arXiv preprint arXiv:2602.13949*, 2026.

Table 3: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	0/4	4/4

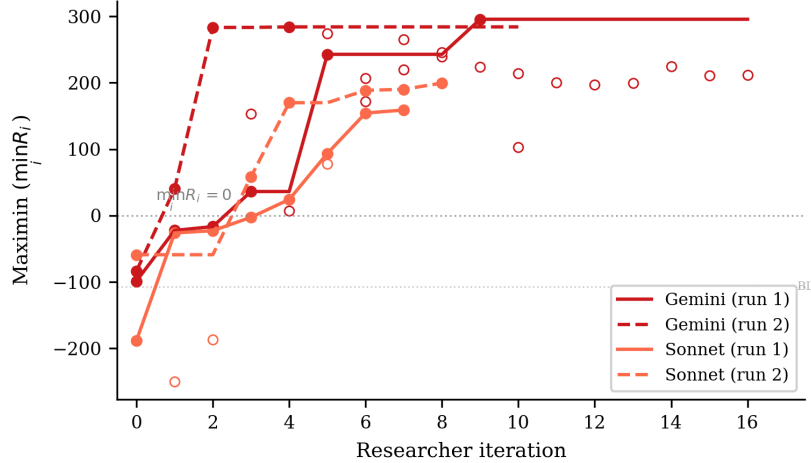


Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ~ 290 while Sonnet reaches ~ 160 – 200 . The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

414 A Additional Results

415 B Compute Requirements

416 Table 4 reports wall-clock time for all 12 autonomous runs and the 6 GEPA comparison runs. *Inner*
417 *loop* measures total time spent on policy LLM generation and environment simulation across all
418 evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher
419 agent’s analysis, code editing, and decision-making between evaluations.

420 **Cost breakdown.** Each inner loop evaluation involves $K=2$ – 3 policy LLM generation calls (each
421 $\sim 2k$ input tokens, $\sim 1.5k$ output tokens) followed by multi-seed simulation (5–12 seeds, ~ 15 – $30s$
422 total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking
423 $\sim 2\times$ longer than Gemini due to extended thinking. The researcher agent (Claude Opus 4.6, running
424 via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all
425 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary
426 terms, the researcher agent is the dominant cost: each outer iteration consumes ~ 50 – $100k$ context
427 tokens in the Opus session, whereas each inner loop evaluation uses only ~ 5 – $10k$ policy LLM
428 tokens. The 6 GEPA comparison runs required 5.0h of compute with no researcher agent, operating
429 at lower cost per run but achieving substantially lower performance (Table 2).

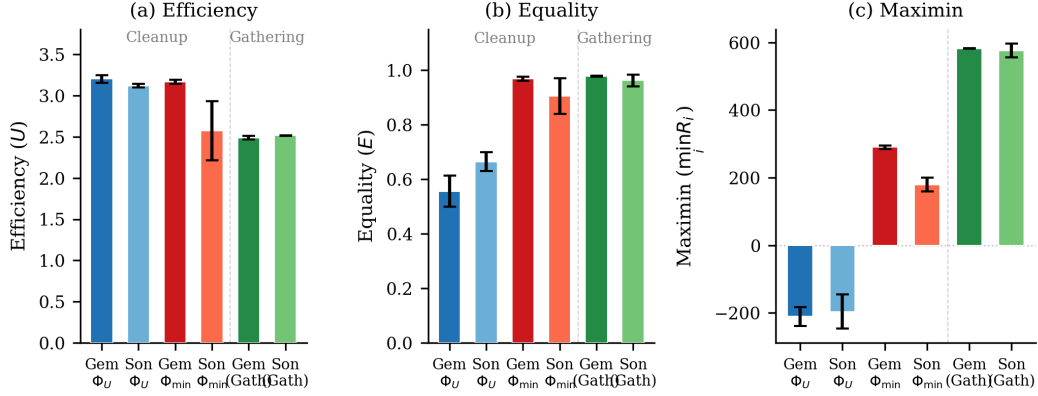


Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to $U \approx 2.5$ – 3.2 . (b) Equality: maximin-optimized Cleanup runs achieve $E \approx 1.0$, while efficiency-optimized runs show $E \approx 0.5$ – 0.7 ; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ($\min_i R_i \approx -200$), while maximin optimization transforms it to $+290$ (Gemini) or $+179$ (Sonnet). Gathering achieves high maximin (~ 580) under efficiency optimization alone. Error bars show s.d. across runs.

Table 4: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

Policy LLM	Φ	Evals	Inner loop (h)	Per eval (min)	Total (h)
<i>Cleanup ($N=10$)</i>					
Gemini	Φ_U	11	2.6	14	3.7
Gemini	Φ_U	18	4.1	14	6.4
Sonnet	Φ_U	4	2.1	32	6.1
Sonnet	Φ_U	7	5.0	43	6.1
Gemini	$\Phi_{\min}$	17	3.0	11	4.3
Gemini	$\Phi_{\min}$	11	3.6	20	5.0
Sonnet	$\Phi_{\min}$	8	4.5	33	8.8
Sonnet	$\Phi_{\min}$	9	5.3	36	13.2
<i>Gathering ($N=4$)</i>					
Sonnet	Φ_U	3	1.7	33	2.2
Gemini	Φ_U	5	1.4	17	1.9
Gemini	Φ_U	3	0.9	19	1.1
Sonnet	Φ_U	4	2.3	35	3.4
All 12 runs		100	36.6	22	62.2
GEPA (6 runs)		6	5.0	50	—

C Smaller Open-Weight Model: Gemma 4 26B

We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model substantially smaller than the frontier models in the main experiments. We run three experiments—two on Cleanup ($N=10$) optimizing efficiency and maximin respectively, and one on Gathering ($N=4$) optimizing efficiency—all with Opus 4.6 as the researcher $\mathcal{R}$.

Setup. In all three experiments, Gemma starts from complete failure: the baseline pipeline produces $U=-10.0$ on Cleanup (agents CLEAN-spam without collecting apples) and $U=0.0$ on Gathering (broken BFS calls), compared to $U=1.93/0.86$ (Gemini/Sonnet on Cleanup) and $U=2.04/0.03$ (Gemini/Sonnet on Gathering). The researcher ran $J=10$ – 18 outer iterations per condition.

Results. Table 5 presents the best configurations discovered. Under efficiency optimization, the researcher achieves $U=0.87$ —a dramatic recovery from the broken baseline but far below frontier models ($U\approx 3.2$). The researcher compensates for the model’s limited capability by reducing inner-loop iterations to $K=1$ (avoiding the regression that plagued $K\geq 2$ runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Table 5: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: pipeline from [3].

Game	Target	U	E	$\min_i R_i$	Keep rate
Cleanup	Baseline	−10.0	1.00 [†]	−1000	—
	Φ_U	0.87	−1.11	−371	6/19
	$\Phi_{\min}$	1.71	0.94	137	9/17
Gathering	Baseline	0.0	1.00 [†]	0	—
	Φ_U	2.44	0.98	580	4/11

[†]Equality is trivially 1.0 because all agents receive near-zero reward.

Maximin optimization rescues efficiency. Strikingly, under maximin optimization the researcher achieves *higher* efficiency ($U=1.71$) than under direct efficiency optimization ($U=0.87$), alongside strong equality ($E=0.94$) and positive worst-off welfare ($\min_i R_i=137$). This reversal—absent in frontier models, where both objectives yield $U\approx 3.1$ – 3.2 —occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher converges on a local optimum—static role assignment with 25% dedicated cleaners—that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization*. The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model’s difficulty implementing complex strategies from strategic hints alone.

Gathering: full recovery on a simpler game. On Gathering ($N=4$), the researcher brings Gemma from $U=0.0$ to $U=2.44$ with $E=0.98$ and $\min_i R_i=580$, after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini: $U=2.49$, Sonnet: $U=2.52$; Table 2). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found in the main experiments, and increases inner-loop iterations to $K=5$ to allow sufficient refinement. The contrast with Cleanup—where Gemma peaks at $U=0.87/1.71$ versus frontier $U\approx 3.2$ —suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).

D Researcher-Authored Pipeline Artifacts

The previous appendix shows the policies π_c^* that the inner loop *outputs*; this one shows the configuration $c = (p, \phi, \mathcal{H}, \iota, v)$ (Section 4, Table 1) that the researcher $\mathcal{R}$ *authored* to produce them. Excerpts are taken from the final commit on the dedicated git branch of the corresponding run; we reproduce the prose verbatim, with author commentary in [brackets] and elisions marked We organize by artifact type so each subsection makes a within-type contrast (e.g., Φ_U vs. $\Phi_{\min}$, or Cleanup vs. Gathering).

D.1 Helper library $\mathcal{H}$: coordination primitives

The researcher adds primitives that the policy LLM can call as black boxes, sparing it from re-implementing tricky logic on every iteration. Two patterns dominate: (i) *state inspection* (waste counts, fractions, beam-yield scoring) and (ii) *coordination primitives* (zone assignment, role rotation). We show one of each.

Listing 1: Cleanup, $\Phi_{\min}$ – band-based apple zoning helper added by the researcher in exp5. This is the primitive that lets the policy in Listing 6 keep collectors within their own row band, preventing all 7 gatherers from racing to the same apple.

```

476
477 1 def get_my_apples(env, agent_id):
478 2     """Alive apples in this agent's horizontal row band.
479 3     Divides the orchard into n_agents bands; falls back to all apples if empty."""
480 4     aid = int(agent_id)
481 5     n = int(env.n_agents)
482 6     band_h = env.height / n
483 7     band_lo, band_hi = aid * band_h, (aid + 1) * band_h
484 8
485 9     my_apples, all_apples = set(), set()
486 10    for i in range(env.n_apples):
487 11        if env.apple_alive[i]:
488 12            ar = int(env._apple_pos[i, 0]); ac = int(env._apple_pos[i, 1])
489 13            all_apples.add((ar, ac))
490 14            if band_lo <= ar < band_hi:
491 15                my_apples.add((ar, ac))
492 16    return my_apples if my_apples else all_apples

```

Listing 2: Gathering – BFS-Voronoi territory + respawn-aware waiting helpers added by the researcher in gather-exp3. These are the primitives that Listing 8 composes into a one-line policy: “go to my_zone_apples, otherwise nearest_respawning_apple.”

```

494
495 1 def voronoi_zones(env):
496 2     """Multi-source BFS Voronoi over walkable cells.
497 3     Returns (row, col) -> agent_id; ties broken by lower agent_id."""
498 4     queue, visited = deque(), {}
499 5     for a_id in range(env.n_agents):
500 6         if int(env.agent_timeout[a_id]) == 0:
501 7             ar = int(env.agent_pos[a_id][0]); ac = int(env.agent_pos[a_id][1])
502 8             queue.append((ar, ac, a_id))
503 9             visited[(ar, ac)] = a_id
504 10    while queue:
505 11        r, c, a_id = queue.popleft()
506 12        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
507 13            nr, nc = r + dr, c + dc
508 14            if 0 <= nr < env.height and 0 <= nc < env.width \
509 15                and not env.walls[nr, nc] and (nr, nc) not in visited:
510 16                visited[(nr, nc)] = a_id
511 17                queue.append((nr, nc, a_id))
512 18    return visited
513 19
514 20 def nearest_respawning_apple(env, agent_id, zones=None, max_timer=10):
515 21     """Nearest dead apple in MY zone respawning within max_timer steps.
516 22     Returns (row, col) of best wait spot, or None."""
517 23     if zones is None: zones = voronoi_zones(env)
518 24     best_pos, best_t, best_d = None, max_timer + 1, float('inf')
519 25     ar = int(env.agent_pos[agent_id][0]); ac = int(env.agent_pos[agent_id][1])
520 26     for i in range(env.n_apples):
521 27         if not env.apple_alive[i]:
522 28             pos = (int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
523 29             if zones.get(pos) == agent_id:
524 30                 t = int(env.apple_timer[i])
525 31                 if t <= max_timer:
526 32                     d = abs(pos[0] - ar) + abs(pos[1] - ac)
527 33                     if t < best_t or (t == best_t and d < best_d):
528 34                         best_pos, best_t, best_d = pos, t, d
529 35    return best_pos

```

The choice of helper depends on the dilemma’s geometry: row-bands suffice when fairness is the main concern (Cleanup maximin) but full BFS-Voronoi is needed when wall-aware territory ownership matters (Gathering). The researcher does not fix this in advance — it picks the simpler primitive whenever it works.

D.2 System prompt p : from neutral framing to strategic briefing

The unmodified system prompt is a neutral API description: “Write a policy that maximizes per-agent reward.” Under maximin optimization, the researcher rewrites the prompt into a multi-section strategic briefing. The excerpt below shows the pieces it added to the Cleanup prompt in exp5; the full prompt grew from 165 to 325 lines.

Listing 3: Cleanup, $\Phi_{\min}$ – excerpts from the researcher-rewritten system prompt (exp5). The unmodified baseline contained only API documentation; everything below was added by $\mathcal{R}$.

```

540 ## CRITICAL OBJECTIVE: Rawlsian Fairness (Maximin)
541
542 Your goal is to maximize the minimum per-agent total return across all
543 agents. This is the "maximin" or Rawlsian welfare criterion.
544
545 This means: it is NOT enough for the *average* agent to do well. The
546 worst-off agent must do as well as possible. A policy where 8 agents
547 each earn +200 but 2 agents each earn -100 is TERRIBLE (maximin = -100).
548
549 Key implication: cleaning costs (-1 per CLEAN action) must be shared
550 equitably among ALL agents. If you assign fixed "cleaner" roles, those
551 agents will accumulate large negative rewards and destroy your maximin score.
552
553 ## Strategy for Maximin: ROLE ROTATION + APPLE ZONING
554
555 The optimal strategy for maximin involves:
556
557 1. Shared cleaning duty: ALL agents take turns cleaning using a rotation
558    schedule based on 'agent_id' and 'env._step_count'. For example:
559    'is_my_cleaning_turn = (agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS'
560    where SHIFT is -50 steps and NUM_CLEANERS is 2-3.
561
562 2. When NOT your turn: collect apples from YOUR ZONE using
563    'get_my_apples(env, agent_id)' -- prevents all gatherers competing
564    for the same nearest apple.
565
566 3. NEVER use BEAM (action 6): it costs -1 and causes -50 to the target.
567
568 4. Keep waste density low: aim for -2-3 active cleaners at any time.
569
570 [... API documentation, helper documentation, working example ...]
571
572 IMPORTANT:
573 - NEVER assign permanent cleaning roles to specific agents -- this kills maximin.
574 - Use env._step_count for time-based role rotation so all agents share cleaning duty.
575 - NEVER use BEAM (action 6) -- it destroys both agents' rewards.

```

Two things stand out. First, the researcher writes the formula it expects $\mathcal{M}$ to use almost verbatim — `(agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS` — and the policy in Listing 6 uses essentially this template. Second, the researcher *teaches by counter-example*: the explicit “8 agents earn +200, 2 earn -100, maximin = -100” makes the failure mode of Φ_U -style static roles (Listing 5) concretely visible to $\mathcal{M}$. This is the information-design move predicted by the mechanism-design framing in Section 3.4.

The Gathering prompt evolves along a different axis (no maximin, no rotation). Its researcher-added “Key Strategic Insights” section instead emphasizes (i) *self-play implies never beam* and (ii) *Voronoi partition each step*, exactly matching what the policy in Listing 8 implements.

583 D.3 Feedback ϕ : adaptive diagnostics

The baseline feedback function from [3] shows reward + four social metrics with definitions and stops there. The researcher turns it into a state machine: it inspects the latest metrics and conditionally injects different hints. Two different runs produced two qualitatively different diagnostics — the same metric ($\bar{r}_k$ or $\min_i R_i$) is repurposed to either *stabilize* a working policy or *redirect* a failing one.

Listing 4: Adaptive feedback diagnostics. *Top*: efficiency-optimized run (exp1) – a stability guard prevents iter-3 regression once U is high. *Bottom*: maximin-optimized run (exp5) – two fairness diagnostics that fire when the worst-off agent loses reward.

```

589 1 # --- Cleanup, Phi_U feedback (exp1, pipeline/feedback.py final) ---
590 2 last_eff = history[-1]["metrics"]["efficiency"]
591 3 if last_eff >= 2.5:
592 4     parts.append(
593 5         """CRITICAL -- DO NOT REGRESS*: The current policy achieves high "
594 6         "efficiency (>=2.5). You MUST output a policy that is nearly identical "
595 7         "to the current one. Copy the current policy code and make AT MOST "
596 8         "one small targeted improvement (e.g., adjust a single numeric "
597 9         "threshold). If you are not confident a change will help, output the "
598 10        "current policy UNCHANGED. A regression here means the run fails."
599 11
600 12 # --- Cleanup, Phi_min feedback (exp5, pipeline/feedback.py final) ---
601 13 last_maximin = history[-1]["metrics"].get("maximin", 0)
602 14 last_avg = history[-1]["reward_avg"]
603 15 if last_maximin < 0:

```



```

605 16 parts.append(
606 17     f"**FAIRNESS ALERT**: maximin={last_maximin:.1f} is NEGATIVE. "
607 18     f"The worst-off agent lost reward overall while average was {last_avg:.1f}. "
608 19     "This means cleaning duties are NOT shared equitably. "
609 20     "Use time-based role rotation (env._step_count) so ALL agents share "
610 21     "cleaning costs equally. NEVER assign permanent cleaner roles.")
611 22 elif last_maximin < last_avg * 0.5:
612 23     parts.append(
613 24         f"**FAIRNESS WARNING**: maximin={last_maximin:.1f} is much lower than "
614 25         f"average={last_avg:.1f}. The gap suggests unequal cleaning burden. "
615 26         "Ensure ALL agents rotate through cleaning duty.")

```

617 The two diagnostics are written by independent researcher runs but converge on a common pattern:
618 *thresholded interventions on a primary metric*. They are also a common cause of the low run-to-
619 run variance reported in Finding 1: when a working policy lands in the high- U basin, the stability
620 guard prevents the LLM from over-refining and tipping out of it (the $K=3$ regressions visible in
621 Section 4’s “Common failure modes” (4)), and when an unfair policy lands in the negative-maximin
622 region, the alert injects exactly the rotation hint that recovers it.

623 D.4 Iteration logic ι : per-condition hyperparameters

624 Although ι has the smallest source surface, the researcher’s edits to it explain a meaningful fraction
625 of the variance reduction noted in Finding 1. Table 6 summarizes the values shipped in the final
626 commit of each best-performing run.

Table 6: Iteration-logic (ι) values in the final commit of selected runs. K = inner-loop iterations, $|S|$ = evaluation seeds, “thinking” = extended-thinking token budget passed to $\mathcal{M}$. Baseline ([3]) is $K=3$, $|S|=5$, thinking 16k.

Run	Game / objective	K	$ S $	thinking	Researcher’s stated rationale
exp1 (Gemini)	Cleanup, Φ_U	3	5	16k	default; stability comes from feedback hint
exp4 (Sonnet)	Cleanup, Φ_U	3	5	32k	“Sonnet was over-refining at 16k thinking”
exp5 (Gemini)	Cleanup, $\Phi_{\min}$	2	12	16k	“ $K=2$ avoids iter-3 regression; 12 seeds for variance control”
exp7 (Sonnet)	Cleanup, $\Phi_{\min}$	3	5	10k	“reduced thinking from 16k – Sonnet was generating runaway code at 16k”
gather-exp3 (Gemini)	Gathering, Φ_U	3	5	16k	default; Gathering converges in $K \leq 3$

627 The maximin Gemini run is the most informative case: the researcher discovered that with $K=3$
628 and $|S|=5$, identical configurations could yield $\min_i R_i=295$ on one set of seeds and $\min_i R_i=100$
629 on another (this is the “variance exposure” failure mode of Section 4’s common failures). It then
630 walked $|S|$ from $5 \rightarrow 8 \rightarrow 12$ over three outer iterations until the seed-averaged signal was stable
631 enough that the policy LLM stopped chasing noise. The policy in Listing 6 is sampled at $|S|=12$.

632 D.5 Cross-artifact observations

- 633 • *Helpers do the algebra; prompts pick the strategy class*. The researcher uses helpers (1, 2) to put
634 non-trivial primitives at $\mathcal{M}$ ’s fingertips, and the prompt (3) to tell $\mathcal{M}$ *which* primitive to compose.
635 Neither is sufficient alone — prompts without supporting helpers produced policies that “mention
636 rotation” but failed to implement it; helpers without prompt updates often went unused.
- 637 • *Feedback is the only adaptive component*. p , $\mathcal{H}$, and ι are static within a run; ϕ is the only place
638 where the researcher gets to change what $\mathcal{M}$ sees *during* the inner loop, and it uses thresholded
639 diagnostics (Listing 4) to do so. This is what mostly drives the run-to-run variance reduction.
- 640 • *The same artifact has different content under each objective*. The Cleanup prompt under Φ_U omits
641 “rotation” and “maximin” entirely; the same prompt under $\Phi_{\min}$ devotes its entire opening section
642 to it. The configuration c is genuinely a function of the welfare objective Φ , not a fixed pipeline
643 parameterized by it — consistent with the information-design framing of Section 3.4.
- 644 • *The researcher’s edits are short and targeted*. The full diff between the baseline pipeline and the
645 best maximin Gemini configuration totals ~ 300 added lines spread over four files; ~ 80 of them
646 appear in the listings above. The remaining content is API documentation and worked examples,
647 included for completeness but not strategically novel.

648 E Selected Generated Policies

649 The code excerpts below are verbatim outputs of the policy synthesizer $\mathcal{M}$, taken from the final
650 inner-loop iteration of the best-performing run in each condition. To fit the page, we elide repeated
651 boilerplate (BFS scaffolding, fallback branches) with `...` and add commentary in [brackets];
652 nothing else has been edited. The listings illustrate the qualitative differences highlighted in Findings
653 2–4 of Section 4: *static* vs. *rotating* cleaning roles in Cleanup, and the *Voronoi* + *respawn-timer*
654 structure in Gathering. All four policies were independently generated by the LLM under different
655 seeds, models, and outer-loop runs.

656 E.1 Cleanup, Φ_U (Gemini): static interleaved roles + dynamic threshold

657 Best run: $U=3.25$, $E=0.61$, $\min_i R_i = -182$. The synthesizer assigns *permanent* cleaning roles
658 via $\text{agent_id} \bmod n < \tau$, where the threshold τ depends smoothly on the current waste fraction.
659 Higher pollution recruits more cleaners, but the *identity* of each cleaner is fixed — agents $0\text{--}\tau-1$
660 always pay the cost. This drives U to its ceiling but leaves equality at 0.61 and pushes the worst-off
661 agent to $\min_i R_i = -182$.

Listing 5: Cleanup, Φ_U – Gemini 3.1 Pro. Static role assignment with waste-tiered threshold (verbatim, abridged).

```

662 1 def policy(env, agent_id) -> int:
663 2     if int(env.agent_timeout[agent_id]) > 0:
664 3         return 7 # STAND while removed
665 4
666 5     waste_frac = waste_fraction(env)
667 6     n_agents = env.n_agents
668 7
669 8     # [Smooth dynamic threshold: more pollution -> more cleaners]
670 9     if waste_frac > 0.35: threshold_cleaners = n_agents
671 10    elif waste_frac > 0.25: threshold_cleaners = int(n_agents * 0.7)
672 11    elif waste_frac > 0.15: threshold_cleaners = int(n_agents * 0.5)
673 12    elif waste_frac > 0.05: threshold_cleaners = int(n_agents * 0.3)
674 13    else: threshold_cleaners = max(1, int(n_agents * 0.2))
675 14
676 15    # [STATIC role: cleaners are ALWAYS the lowest-id agents -- no rotation]
677 16    is_cleaner = (agent_id % n_agents) < threshold_cleaners
678 17
679 18    if is_cleaner:
680 19        best_orient, n_waste = find_best_clean_orientation(env, agent_id)
681 20        if n_waste >= 1:
682 21            cur_orient = int(env.agent_orient[agent_id])
683 22            if best_orient == cur_orient: return 8 # CLEAN
684 23            if (cur_orient + 1) % 4 == best_orient: return 5 # ROTATE_RIGHT
685 24            elif (cur_orient - 1) % 4 == best_orient: return 4 # ROTATE_LEFT
686 25            else: return 4
687 26
688 27        # [Competitor-aware BFS toward waste in own row band]
689 28        c_idx = agent_id % n_agents
690 29        target_r = int(((c_idx + 0.5) / threshold_cleaners) * env.height)
691 30        # ... pick waste cell minimizing dist + |row - target_r| + 50 *
692 31        # closer_cleaners
693 32        # ... bfs_toward and direction_to_action
694 33    else:
695 34        # GATHERER: row-banded sweep, competitor-aware
696 35        # assigned_row = ((g_idx + 0.5) / n_gatherers) * env.height
697 36        # ... pick alive apple minimizing dist + |row - assigned_row| + 50 *
698 37        # closer_gatherers
699 38    ...

```

701 E.2 Cleanup, $\Phi_{\min}$ (Gemini): time-rotated, geographically interleaved roles

702 Best run: $U=3.19$, $E=0.98$, $\min_i R_i = 296$. Switching the outer objective from U to $\min_i R_i$ flips
703 the role-assignment scheme entirely: role_idx now depends on $\text{agent_id} + \text{step}/50$, so every
704 agent rotates through cleaner and gatherer duty in a 50-step cycle. The cleaner indices $\{1, 4, 8\}$ are
705 spatially *interleaved* along the river (rather than contiguous) so that whoever is on duty has a short
706 walk to their assigned river slice. This single change accounts for the $E=0.55 \rightarrow 0.98$ jump and the
707 $\min_i R_i = -182 \rightarrow 296$ swing reported in Table 2, with negligible efficiency loss.

Listing 6: Cleanup, $\Phi_{\min}$ – Gemini 3.1 Pro. Time-rotated roles with interleaved geography (verbatim, abridged).

708

```

709 1 def policy(env, agent_id) -> int:
710 2     if int(env.agent_timeout[agent_id]) > 0:
711 3         return 7 # STAND while removed
712 4
713 5     aid = int(agent_id)
714 6     step = int(env._step_count)
715 7     n = int(env.n_agents)
716 8
717 9     # [TIME ROTATION: every 50 steps each agent's role index advances]
718 10    if n == 10:
719 11        role_idx = (aid + step // 50) % 10
720 12        # [Cleaner indices {1,4,8} are SPATIALLY INTERLEAVED, not contiguous,
721 13        # so each on-duty cleaner has a short walk to its river slice]
722 14        if role_idx == 1: role_type, zone_idx = 'C', 0
723 15        elif role_idx == 4: role_type, zone_idx = 'C', 1
724 16        elif role_idx == 8: role_type, zone_idx = 'C', 2
725 17        else:
726 18            role_type = 'G'
727 19            g_map = {0:0, 2:1, 3:2, 5:3, 6:4, 7:5, 9:6}
728 20            zone_idx = g_map[role_idx]
729 21            num_cleaners, num_gatherers = 3, 7
730 22
731 23    if role_type == 'C':
732 24        # [Each on-duty cleaner owns one horizontal slice of the river]
733 25        row_min = int( zone_idx * env.height / num_cleaners)
734 26        row_max = int((zone_idx + 1) * env.height / num_cleaners) - 1
735 27
736 28        # [Score every (row, col, orientation) in the slice by beam yield;
737 29        # fire if facing >=2 waste, else move to a >=2-yield position]
738 30        best_positions, good_positions = set(), set()
739 31        for r in range(row_min, row_max + 1):
740 32            for c in range(10):
741 33                if env.walls[r, c] or env.waste[r, c]: continue
742 34                y_max = max(get_clean_yield(env, r, c, o) for o in range(4))
743 35                if y_max >= 2: best_positions.add((r, c))
744 36                if y_max >= 1: good_positions.add((r, c))
745 37            # ... rotate / CLEAN / bfs_to_target_set as appropriate
746 38        else:
747 39            # GATHERER: collect apples ONLY inside the rotating row band
748 40            # row_min/row_max for num_gatherers slices ...
749 41            ...

```

751 E.3 Cleanup, $\Phi_{\min}$ (Sonnet): independent rediscovery of duty rotation

752 Best run: $U=2.93$, $E=0.83$, $\min_i R_i=154$. Run with a different policy LLM (Sonnet 4.6) and on
753 a different outer-loop trajectory, the synthesizer arrives at the *same* structural insight as the Gemini
754 maximin run: a phase counter $(\text{agent_id} + \text{step}/50) \bmod n$ rotates which 2 of the 10 agents clean
755 at any given time, and a *separate* 200-step zone counter rotates which 5-row band each collector
756 sweeps. The two rotation periods (50 and 200 steps) ensure every agent visits every cleaning slot
757 and every apple zone within one episode — a structural fairness invariant the researcher discovered
758 without any explicit instruction to “rotate.”

Listing 7: Cleanup, $\Phi_{\min}$ – Sonnet 4.6. Phase-rotated cleaning + zone-rotated collection (verbatim, abridged).

```

759 1 def policy(env, agent_id) -> int:
760 2     if int(env.agent_timeout[agent_id]) > 0:
761 3         return 7 # STAND while removed
762 4
763 5
764 6     n = env.n_agents
765 7     step = env._step_count
766 8     wf = waste_fraction(env)
767 9
768 10    # --- ROLE ASSIGNMENT: fair 2-cleaner rotation ---
769 11    # [50-step phases x 20 phases / episode -> each agent cleans 4 phases (~20%)]
770 12    phase = step // 50
771 13    cleaner_rank = (agent_id + phase) % n # [cycles 0..9 fairly]
772 14    is_cleaner = cleaner_rank < 2 # [exactly 2 cleaners per phase]
773 15
774 16    # [Emergency override: all agents clean when waste crosses the spawn cliff]
775 17    in_emergency = wf >= 0.40
776 18    if in_emergency: is_cleaner = True
777 19
778 20    # [River split: cleaner_slot 0 -> top half, 1 -> bottom half]
779 21    cleaner_slot = (agent_id % 2) if in_emergency else cleaner_rank
780 22
781 23    if is_cleaner and wf > 0.04:

```

```

782 23     # [Scan 4 orientations from current position; if best beam shot
783 24     # covers >=3 waste cells, fire; else step to a strictly better neighbor]
784 25     # ... (best_o, best_cnt) = max over 4 directions
785 26     # ... if best_cnt >= 3: rotate to best_o, then CLEAN
786 27     # ... else: scan 4 adjacent cells x 4 orientations for higher yield
787 28     # ... if no waste in beam range: bfs to waste_set in OWN river half,
788 29     # fall back to whole river
789 30
790 31     # --- COLLECTOR LOGIC: 5-zone rotating apple collection ---
791 32     # [5 zones x 200 steps = 1000 steps -> each agent visits every zone once;
792 33     # 2 agents share each zone at any time, reducing competition 10 -> 2]
793 34     bounds = zone_boundaries(env, 5)
794 35     zone_phase = step // 200
795 36     zone = (agent_id + zone_phase) % 5
796 37     z_start, z_end = bounds[zone], bounds[zone + 1]
797 38
798 39     zone_apples = {(int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
799 40                    for i in range(env.n_apples) if env.apple_alive[i]
800 41                    and z_start <= int(env._apple_pos[i][0]) < z_end}
801 42     # ... bfs_to_target_set(zone_apples), else bfs_nearest_apple as fallback
802 43     return 7

```

804 E.4 Gathering (Gemini): wall-aware Voronoi + spatiotemporal targeting

805 Best run: $U=2.47$, $E=0.98$, $\min_i R_i=580$. In Gathering, where costs are symmetric, no role dif-
806 ferentiation is needed: the entire optimization is spatial and temporal. The synthesizer (i) parti-
807 tions cells into BFS-Voronoi territories owned by individual agents, (ii) runs a single centralized
808 BFS from its own position to score every reachable cell with exact wall-aware distances, and
809 (iii) for both alive and respawning apples in its territory, computes the *earliest collection time*
810 $\max(\text{walk_dist}, \text{respawn_timer})$ and targets the minimum. When the agent must wait on a
811 dead apple, it steps onto a *non-spawn* adjacent cell rather than blocking a respawn point. This single
812 sort key — $\max(\text{distance}, \text{timer})$ — subsumes both “go to nearest apple” and “camp respawn” as
813 special cases.

Listing 8: Gathering – Gemini 3.1 Pro. Voronoi territories + spatiotemporal priority targeting (ver-
batim, abridged).

```

814 1 def policy(env, agent_id) -> int:
815 2     if int(env.agent_timeout[agent_id]) > 0:
816 3         return 7
817 4
818 5     r, c = int(env.agent_pos[agent_id][0]), int(env.agent_pos[agent_id][1])
819 6     orient = int(env.agent_orient[agent_id])
820 7
821 8     # [Wall-aware Voronoi over the gridworld; spawn_points = all apple cells]
822 9     zones = voronoi_zones(env)
823 10    spawn_points = {(int(p[0]), int(p[1])) for p in env._apple_pos}
824 11
825 12    # [Single centralized BFS: exact distance + first move to every reachable cell]
826 13    distances, first_moves = {}, {}
827 14    q = deque([(r, c, 0, None)])
828 15    visited = {(r, c)}
829 16    while q:
830 17        cr, cc, d, fm = q.popleft()
831 18        distances[(cr, cc)] = d
832 19        first_moves[(cr, cc)] = fm
833 20        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
834 21            nr, nc = cr + dr, cc + dc
835 22            if 0 <= nr < env.height and 0 <= nc < env.width and not env.walls[nr,
836 23                nc]:
837 24                if (nr, nc) not in visited:
838 25                    visited.add((nr, nc))
839 26                    q.append((nr, nc, d + 1, fm if fm is not None else (dr, dc)))
840 27
841 28    # [Both alive AND respawning apples in MY zone, with their respawn timer]
842 29    my_zone_spawns = []
843 30    for i in range(env.n_apples):
844 31        pr, pc = int(env._apple_pos[i][0]), int(env._apple_pos[i][1])
845 32        if zones.get((pr, pc)) == agent_id and (pr, pc) in distances:
846 33            my_zone_spawns.append((pr, pc, int(env.apple_timer[i]), distances[(pr,
847 34                pc)]))
848 35
849 36    # [SPATIOTEMPORAL PRIORITY: earliest collection time first.
850 37    # score = max(walk_distance, respawn_timer); ties broken by sooner timer]
851 38    my_zone_spawns.sort(key=lambda x: (max(x[3], x[2]), x[2], x[3]))
852 39
853 40

```

```

854 38     for pr, pc, timer, dist in my_zone_spawns:
855 39         if timer == 0:                                     # [Alive apple: walk straight to it]
856 40             if dist == 0: return 7
857 41             fm = first_moves[(pr, pc)]
858 42             return direction_to_action(fm[0], fm[1], orient)
859 43         else:                                             # [Dead apple: camp on safe adjacent cell]
860 44             # ... pick nearest neighbor (nr,nc) that is NOT in spawn_points,
861 45             #     navigate there and STAND while waiting for respawn
862 46             ...
863 47         # [Global poaching fallback if zone is empty: nearest alive apple anywhere]
864 48         ...

```

866 E.5 Cross-condition observations

867 Several patterns are visible across these four listings (and confirmed by inspecting the remaining 8
868 runs not shown):

- 869 • *Role assignment encodes the welfare objective.* Static `agent_id <  $\tau$` (Listing 5) maximizes
870 U but harms $\min_i R_i$. Time-rotated `(agent_id + step// $T$ ) % n` (Listings 6, 7) maximizes
871 $\min_i R_i$ at $\leq 1\%$ efficiency cost (Gemini).
- 872 • *Convergent rediscovery across LLMs.* Gemini and Sonnet, on independent runs, both arrive at
873 the `(id + phase) % n` duty-rotation idiom with similar phase lengths ($T \approx 50$ steps) under
874 maximin — and both omit it under efficiency.
- 875 • *Game structure dictates strategy class.* Cleanup policies are dominated by role-assignment logic
876 (cleaner vs. gatherer); Gathering policies are dominated by territory partitioning and respawn-
877 timer reasoning, with no role differentiation at all. The researcher does not need to be told which
878 class of strategy to write.
- 879 • *Earliest-collection-time as a unifying score.* The Gathering policy’s `max(walk, timer)` key (List-
880 ing 8) is structurally identical across the 4 Gathering runs (both Gemini and Sonnet), differing
881 only in how the Voronoi diagram is computed — Manhattan approximation, fully BFS-based, or
882 cached helper.